

Software & Tools

Hacking DVI files: Birth of DViasm

Jin-Hwan Cho

Abstract

This paper is devoted to the first step of developing a new DVI editing utility, called DViasm. Editing DVI files consists of three parts: disassembling, editing, and assembling. DViasm disassembles a DVI file to a human-readable text format (more flexible than DTL), and assembles the output back to a DVI file.

DViasm is useful for people who have a DVI file without $\text{\TeX}$ source, but need to modify the document. It enables us to put a preprint number, a watermark, or an emblem on the document without touching the $\text{\TeX}$ source. DViasm is attractive to even a $\text{\TeX}$ expert who wants to modify a few words in his document more than a hundred pages long.

We discuss in the paper how DViasm supplements $\text{\TeX}$. The current version supports only the standard DVI file format as in DVItyp and DTL. The next versions will support 16-bit $\text{\TeX}$ extensions including Omega, p $\text{\TeX}$, and X $\text{\TeX}$.

1 Introduction

Have you ever heard of DVI, not the Digital Visual Interface¹ but the DeVice-Independent file format? In past years, every $\text{\TeX}$ user knew what DVI is and used DVI utilities to view and print $\text{\TeX}$ results. However, in recent times, $\text{\TeX}$ users have paid attention to DVI less and less because pdf $\text{\TeX}$ outputs directly to the PDF² file format. It is true without doubt that PDF is more powerful than DVI in almost all aspects. Then, do we have to obsolete DVI as PostScript is gradually replaced by PDF?

The DVI file format was designed by David R. Fuchs in 1979, in contrast to the release of PDF version 1.0 in 1993. It is intended to be both compact and easily interpreted by a machine [4, §14]. The most powerful aspect of DVI compared to PDF is

nothing but *simplicity*. Imagine the speed of three previewers of DVI, PostScript, and PDF, and compare also the file size of the three different file formats. Furthermore, simplicity enables us to control DVI files in various ways. One of these is to edit DVI files directly—the main object of this paper.

There are many applications of editing DVI files. The most critical situation is when we have a DVI file without $\text{\TeX}$ source, but we want to modify or to add something to the document. A technical editor may want to put a preprint number on each paper without touching the $\text{\TeX}$ source. He may also want to put a watermark or an emblem on every paper.

Editing a DVI file is much faster for a $\text{\TeX}$ novice than learning $\text{\TeX}$, when all he wants to do is to add some decorations to his document, and is not familiar with $\text{\TeX}$ codes. It may even be attractive to a $\text{\TeX}$ expert who wants to modify a few words in a long document.

Since a DVI file consists of binary data, it must be converted to a human readable format to inspect and edit its contents. The original DVI utility is DVItyp [4], written by Donald E. Knuth in 1982. It has two chief purposes: to validate DVI files, and to give an example for developers of DVI utilities [4, §1]. DVItyp is a nice utility to inspect the contents of a DVI file because of its human readable text output. However, it lacks any procedure for converting the output back to a DVI file.

A true DVI editing utility is the Device-independent Text Language (DTL) package [5] developed by Geoffrey Tobin. It includes two utilities `dv2dt` and `dt2dv` for converting from DVI to DTL and vice versa. It is notable that there is a one-to-one correspondence between DTL and DVI, and that DTL does not require TFM (font metric) files, in contrast to DVItyp. However, DTL is not flexible for ordinary $\text{\TeX}$ users. For example, users must choose the correct command from `r1` to `r4` according to the width of the move to the right. Moreover, the latest version of DTL was released in 1995, and so it does not support extended DVI formats generated by Omega³ or Japanese p $\text{\TeX}$.⁴

The development plan for a new DVI editing utility, called DViasm, consists of three phases. This paper is devoted to the first step, where DViasm is introduced with several examples. The current version of DViasm supports only the standard DVI file format, like DVItyp and DTL, but is more flexible than DTL.

Editor's note: Reprinted from *The Prac $\text{\TeX}$ Journal* 2007-1 (<http://tug.org/pracjourn>), by permission.

¹ A video interface standard designed to maximize the visual quality of digital display devices such as flat panel LCD computer displays and digital projectors [Wikipedia, <http://en.wikipedia.org/wiki/DVI>].

² PDF (Portable Document Format) is an open file format created and controlled by Adobe Systems, for representing two-dimensional documents in a device independent and resolution independent fixed-layout document format [Wikipedia, <http://en.wikipedia.org/wiki/PDF>].

³ An extension of $\text{\TeX}$ by John Plaice and Yannis Haralambous, <http://omega.enstb.org>.

⁴ ASCII Nihongo $\text{\TeX}$ by ASCII Corporation, <http://www.ascii.co.jp/pb/ptex/index.html>.